

Python - Directories

Directories in Python

In Python, directories, commonly known as folders in operating systems, are locations on the filesystem used to store files and other directories. They serve as a way to group and manage files hierarchically.

Python provides several modules, primarily **os** and **os.path**, along with `shutil`, that allows you to perform various operations on directories.

These operations include creating new directories, navigating through existing directories, listing directory contents, changing the current working directory, and removing directories.

Checking if a Directory Exists

Before performing operations on a directory, you often need to check if it exists. We can check if a directory exists or not using the **os.path.exists()** function in Python.

This function accepts a single argument, which is a string representing a path in the filesystem. This argument can be –

- **Relative path** – A path relative to the current working directory.
- **Absolute path** – A complete path starting from the root directory.

Example

In this example, we check whether the given directory path exists using the `os.path.exists()` function –

```
import os

directory_path = "D:\\Test\\MyFolder\\"

if os.path.exists(directory_path):
    print(f"The directory '{directory_path}' exists.")
else:
    print(f"The directory '{directory_path}' does not exist.")
```

Following is the output of the above code –

```
The directory 'D:\\Test\\MyFolder\\' exists.
```

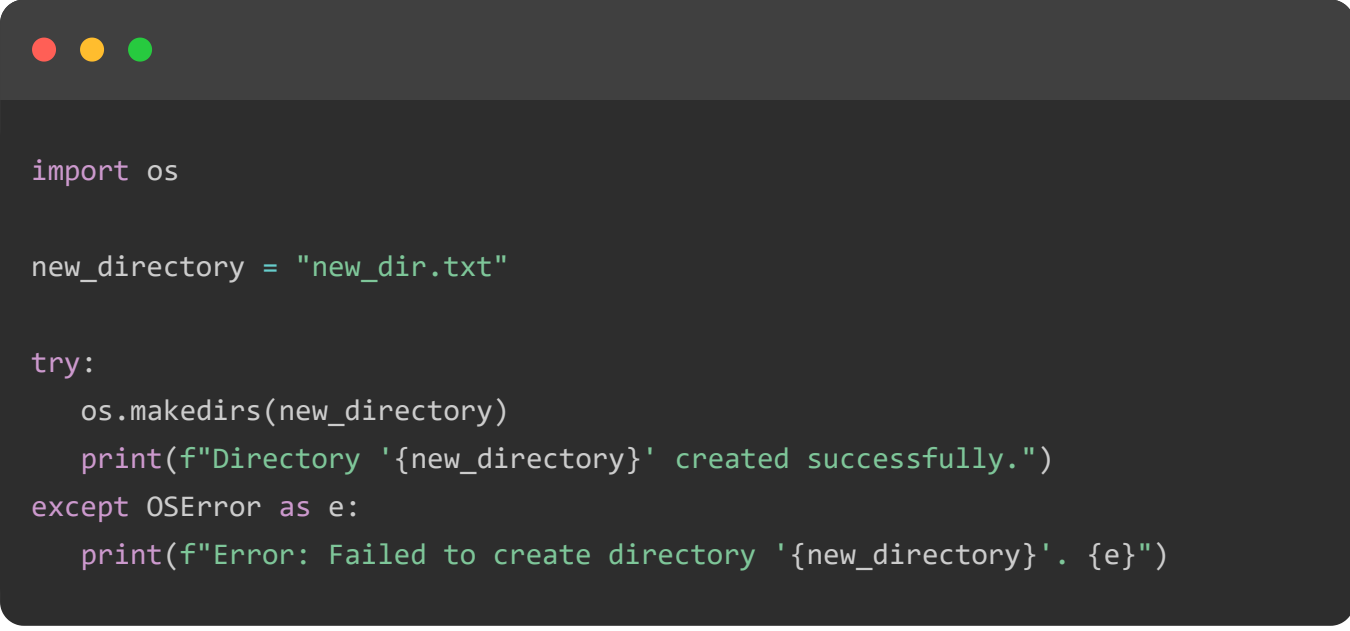
Creating a Directory

You create a new directory in Python using the **os.makedirs()** function. This function creates intermediate directories if they do not exist.

The `os.makedirs()` function accepts a "path" you want to create as an argument. It optionally accepts a "mode" argument that specifies the permissions to set for the newly created directories. It is an integer, represented in octal format (e.g., `0o755`). If not specified, the default permissions are used based on your system's umask.

Example

In the following example, we are creating a new directory using the `os.makedirs()` function –



```
import os

new_directory = "new_dir.txt"

try:
    os.makedirs(new_directory)
    print(f"Directory '{new_directory}' created successfully.")
except OSError as e:
    print(f"Error: Failed to create directory '{new_directory}'. {e}")
```

After executing the above code, we get the following output –

```
Directory 'new_dir.txt' created successfully.
```

The mkdir() Method

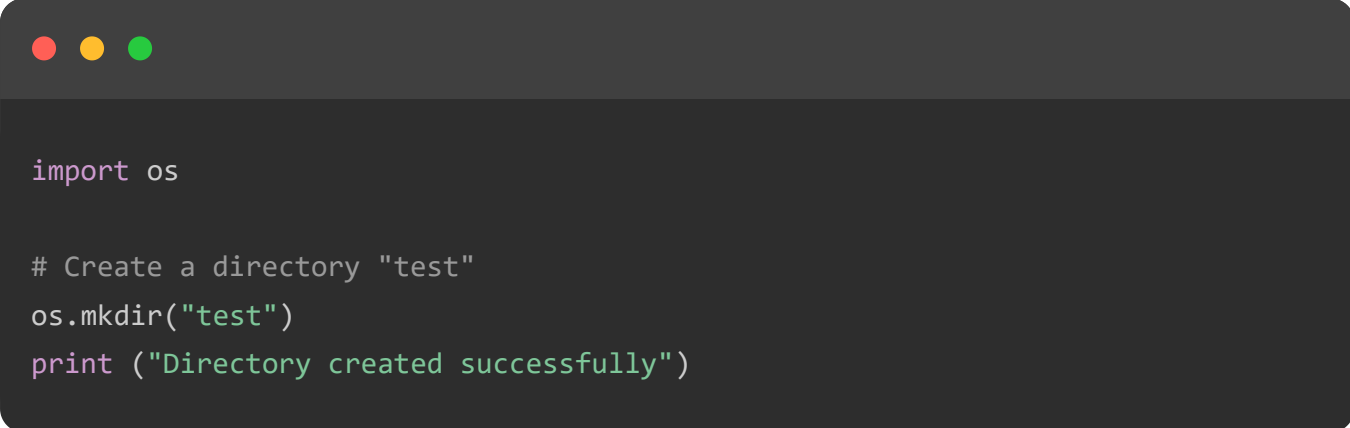
You can use the **mkdir()** method of the `os` module to create directories in the current directory. You need to supply an argument to this method, which contains the name of the directory to be created.

Following is the syntax of the **mkdir()** method in Python –

```
os.mkdir("newdir")
```

Example

Following is an example to create a directory test in the current directory –

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. It contains the following Python code:

```
import os

# Create a directory "test"
os.mkdir("test")
print ("Directory created successfully")
```

The result obtained is as shown below –

```
Directory created successfully
```

Get Current Working Directory

To retrieve the current working directory in Python, you can use the **os.getcwd()** function. This function returns a string representing the current working directory where the Python script is executing.

Syntax

Following is the basic syntax of the **getcwd()** function in Python –

```
os.getcwd()
```

Example

Following is an example to display the current working directory using the `getcwd()` function –

```
import os

current_directory = os.getcwd()
print(f"Current working directory: {current_directory}")
```

We get the output as follows –

```
Current working directory: /home/cg/root/667ba7570a5b7
```

Listing Files and Directories

You can list the contents of a directory using the `os.listdir()` function. This function returns a list of all files and directories within the specified directory path.

Example

In the example below, we are listing the contents of the specified directory path using the **`listdir()`** function –

```
import os

directory_path = r"D:\MyFolder\Pictures"

try:
    contents = os.listdir(directory_path)
    print(f"Contents of '{directory_path}':")
    for item in contents:
        print(item)
except OSError as e:
```

```
print(f"Error: Failed to list contents of directory '{directory_path}'.  
{e}")
```

Output of the above code is as shown below –

```
Contents of 'D:\MyFolder\Pictures':  
Camera Roll  
desktop.ini  
Saved Pictures  
Screenshots
```

Changing the Current Working Directory

You can change the current directory using the **chdir()** method. This method takes an argument, which is the name of the directory that you want to make the current directory.

Syntax

Following is the syntax of the chdir() method in Python –

```
os.chdir("newdir")
```

Example

Following is an example to change the current directory to Desktop using the chdir() method –

```
import os  
  
new_directory = r"D:\MyFolder\Pictures"  
  
try:  
    os.chdir(new_directory)  
    print(f"Current working directory changed to '{new_directory}'.")  
except OSError as e:  
    print(f"Error: Failed to change working directory to '{new_directory}'.  
{e}")
```

We get the output as shown below –

Current working directory changed to 'D:\MyFolder\Pictures'.

Removing a Directory

You can remove an empty directory in Python using the **os.rmdir()** method. If the directory contains files or other directories, you can use **shutil.rmtree()** method to delete it recursively.

Syntax

Following is the basic syntax to delete a directory in Python –

```
os.rmdir(directory_path)
# or
shutil.rmtree(directory_path)
```

Example

In the following example, we remove an empty directory using the os.rmdir() method –

```
import os
directory_path = r"D:\MyFolder\new_dir"

try:
    os.rmdir(directory_path)
    print(f"Directory '{directory_path}' successfully removed.")
except OSError as e:
    print(f"Error: Failed to remove directory '{directory_path}'. {e}")
```

It will produce the following output –

Directory 'D:\MyFolder\new_dir' successfully removed.

TOP TUTORIALS

Python Tutorial

Java Tutorial

C++ Tutorial
C Programming Tutorial
C# Tutorial
PHP Tutorial
R Tutorial
HTML Tutorial
CSS Tutorial
JavaScript Tutorial
SQL Tutorial

TRENDING TECHNOLOGIES

Cloud Computing Tutorial
Amazon Web Services Tutorial
Microsoft Azure Tutorial
Git Tutorial
Ethical Hacking Tutorial
Docker Tutorial
Kubernetes Tutorial
DSA Tutorial
Spring Boot Tutorial
SDLC Tutorial
Unix Tutorial

CERTIFICATIONS

Business Analytics Certification
Java & Spring Boot Advanced Certification
Data Science Advanced Certification
Cloud Computing And DevOps
Advanced Certification In Business Analytics



Chapters ▾

Categories ≡

Game Development Certification
Front-End Developer Certification
AWS Certification Training
Python Programming Certification

COMPILERS & EDITORS

[Online Java Compiler](#)

[Online Python Compiler](#)

[Online Go Compiler](#)

[Online C Compiler](#)

[Online C++ Compiler](#)

[Online C# Compiler](#)

[Online PHP Compiler](#)

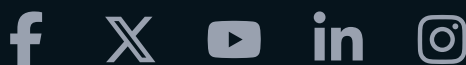
[Online MATLAB Compiler](#)

[Online Bash Terminal](#)

[Online SQL Compiler](#)

[Online Html Editor](#)

[ABOUT US](#) | [OUR TEAM](#) | [CAREERS](#) | [JOBS](#) | [CONTACT US](#) | [TERMS OF USE](#) |
[PRIVACY POLICY](#) | [REFUND POLICY](#) | [COOKIES POLICY](#) | [FAQ'S](#)



Tutorials Point is a leading Ed Tech company striving to provide the best learning material on technical and non-technical subjects.

© Copyright 2026. All Rights Reserved.